

GUILDA 3 — O ARQUIVO ARCANO: MÓDULO DE TABELAS HASH

Biblioteca de Brasilândia — Acesso Direto e Tratamento de Colisões

Instituição: UNIJUÍ — Ciência da Computação / Eng. Software	Professor: Marcos Ronaldo Melo Cavalheiro
Módulo: Tabela Hash (Aula 06 - Unidade 1)	Autores: João Alfredo Williges Cunha, João Victor Benetti Filipim, João Pedro Mussi de Medeiros e Mauro Pezzetta Roncata
Linguagem: Java 21 (IntelliJ IDEA)	Data de Entrega: Aula 8 — Setembro / 2026

1. INTRODUÇÃO E CENÁRIO LÚDICO

No universo narrativo do reino de Brasilândia, a Biblioteca imperial guarda artefatos e pergaminhos sagrados em suas galerias arcanas. Após o ataque do vilão *NullPointer*, a ordem de catalogação das galerias foi corrompida. A missão da **Guilda 3 (O Arquivo Arcano)** é reconstruir o sistema de busca de acesso direto por meio de uma **Tabela Hash (Tabela de Dispersão)** implementada integralmente do zero em Java 21, sem a utilização de coleções prontas (como `java.util.HashMap`).

Cada pergaminho possui um identificador numérico único (id) utilizado como chave de busca. A tabela hash transforma essa chave em um endereço direto dentro do vetor principal. Pelo *Paradoxo do Aniversário*, colisões são estatisticamente inevitáveis. Para gerenciar os conflitos, a Guilda 3 implementou a estratégia de **Encadeamento Separado (Separate Chaining)** com listas encadeadas simples e modelou teoricamente a estratégia alternativa de **Endereçamento Aberto por Sondagem Linear**.

Requisitos do Projeto

Diretrizes Técnicas Obrigatórias do Edital:

- Implementação própria de estruturas dinâmicas sem coleções pré-prontas do Java;
- Tratamento de colisões por encadeamento separado com a classe `NoHash`;
- Verificação rigorosa contra inserção de chaves duplicadas na tabela;
- Medição transparente do desempenho com contagem exata de comparações na busca;
- Monitoramento contínuo do Fator de Carga ($\alpha = N / M$) e religamento seguro de ponteiros na remoção.

2. MODELAGEM DE DADOS E TIPOS ABSTRATOS (TADS)

A arquitetura do sistema é dividida em três componentes modulares no padrão orientado a objetos:

Pergaminho	model	Entidade de dados do domínio. Armazena id (chave), título, categoria e poderMístico.
NoHash	hashtable	Nó da lista encadeada simples. Contém o objeto Pergaminho e a referência para o próximo nó.

GUILDA 3: O ARQUIVO ARCANO — MANUAL DO DESENVOLVEDOR ESTRUTURA DE DADOS

TabelaHash	hashtable	TAD da Tabela Hash com vetor de M posições, função $h(k) = k \bmod M$, inserção, busca, remoção e cálculo do fator de carga.
------------	-----------	---

Main	main	Classe executável do Desafio Lúdico. Executa testes de colisão, duplicatas, buscas e remoção intermediária.
------	------	---

Função de Dispersão (Hashing por Divisão):

A transformação de chave adotada utiliza a operação do resto da divisão com proteção contra inteiros

negativos: $h(chave) = |chave| \bmod M$

Onde M representa o tamanho do vetor da tabela (fixado em $M = 7$, um número primo, garantindo a dispersão uniforme e facilitando as colisões intencionais do teste).

3. ESTRATÉGIA IMPLEMENTADA: ENCADEAMENTO SEPARADO

No **Encadeamento Separado (Separate Chaining)**, o vetor principal da tabela de tamanho M armazena referências para o início de listas simplesmente encadeadas (instâncias de NoHash). Quando duas ou mais chaves colidem no mesmo índice calculado por $h(k)$, os elementos são encadeados no início da lista daquela posição ($O(1)$).

Algoritmos das Operações Principais (Java):

<pre>// Inserção com verificação de duplicada e encadeamento no início O(1) public boolean inserir(Pergaminho pergaminho) { if (pergaminho == null) return false; int indice = h(pergaminho.getId()); NoHash atual = tabela[indice]; while (atual != null) { if (atual.getPergaminho().getId() == pergaminho.getId()) { System.out.println("ERRO!!!: Chave " + pergaminho.getId() + " já existe!"); return false; } atual = atual.getProximo(); } NoHash novo = new NoHash(pergaminho); novo.setProximo(tabela[indice]); // Push no início O(1) tabela[indice] = novo; quantidade++; return true; }</pre>	<pre>// Remoção por chave com religamento de ponteiros anterior e atual public boolean remover(int id) { int indice = h(id); NoHash atual = tabela[indice]; NoHash anterior = null; while (atual != null) { if (atual.getPergaminho().getId() == id) { if (anterior == null) tabela[indice] = atual.getProximo(); else anterior.setProximo(atual.getProximo()); quantidade--; return true; } anterior = atual; atual = atual.getProximo(); } return false; }</pre>
---	--

4. ESTRATÉGIA NÃO ESCOLHIDA: ENDEREÇAMENTO ABERTO (SONDAGEM LINEAR)

Conforme exigido pelas normas do edital, a estratégia de **Endereçamento Aberto por Sondagem Linear**, embora não tenha sido a selecionada para a implementação em Java, foi devidamente analisada e especificada nesta documentação.

Mecanismo de Funcionamento:

No endereçamento aberto, todos os registros residem diretamente dentro do vetor principal da tabela (sem listas externas). Quando ocorre colisão na posição inicial $h(k) = |k| \bmod M$, o algoritmo testa sequencialmente as posições subsequentes:

$h(k, j) = (h(k) + j) \bmod M$, onde $j = 0, 1, 2, \dots, M-1$

A tabela opera como uma fila circular graças à aritmética modular. Se uma posição estiver ocupada, incrementa-se j até

encontrar um slot livre.

Por que a 'Lápide' é Obrigatória em Sondagem Linear?

O Conceito Crítico da 'Lápide' (Tombstone / Marcador REMOVIDO):

A exclusão de um elemento em endereçamento aberto apresenta uma complicação grave: se uma posição intermediária de uma cadeia de sondagem for simplesmente limpa (marcada como VAZIA), buscas posteriores por elementos que colidiram e foram gravados à frente serão interrompidas prematuramente, retornando incorretamente 'elemento não encontrado'.

Solução: Utiliza-se um marcador especial chamado **Lápide (REMOVIDO)**. Durante a busca, a varredura continua ao encontrar uma lápide, parando apenas ao localizar a chave ou encontrar um slot genuinamente VAZIO. Durante a inserção, posições com lápides podem ser reaproveitadas.

Pseudocódigo da Busca com Lápide em Sondagem Linear:

```
função BUSCAR_SONDAGEM_LINEAR(tabela, M, chave):
    i = Math.abs(chave) mod M
    tentativas = 0
    enquanto estado[i] != VAZIO e tentativas < M faça:
        se estado[i] == OCUPADO e chaves[i] == chave então:
            retornar valores[i] // Encontrado com sucesso
        fim se
        i = (i + 1) mod M // Avança circularmente mesmo se for REMOVIDO (Lápide)
        tentativas = tentativas + 1
    fim enquanto
    retornar NULO // Parou em VAZIO: elemento não existe na tabela
fim função
```

5. RESULTADOS DO DESAFIO LÚDICO E VALIDAÇÃO EXPERIMENTAL

O programa principal (Main.java) executou a simulação completa do **Desafio Lúdico da Guilda 3** em uma tabela de tamanho \$M = 7\$ (número primo). A execução validou todas as operações obrigatórias com total exatidão:

1. Estado Inicial (M=7)	Tabela inicializada inteiramente nula. Exibição de slots `[0]` a `[6]` como `[VAZIO]!!!`. Fator de carga \$\alpha = 0,00\$.
2. Colisões Intencionais	Inserção dos IDs `7`, `14` e `21` (todos \$\text{mod } 7 = 0\$) no slot `0`, formando a cadeia `21 -> 14 -> 7 -> null`. Inserção dos IDs `10` e `17` (ambos \$\text{mod } 7 = 3\$) no slot `3`, formando `17 -> 10 -> null`. Fator de carga subiu para \$\alpha = 0,71\$ (5/7).
3. Chave Duplicada	Tentativa de re-inserir o ID `14`. O sistema varreu o slot `0`, detectou a duplicação e barrou a operação disparando a mensagem de erro: `ERRO!!!: Chave 14 já existe na tabela!!!`.
4. Medição de Comparações	- ID 21 (cabeça do slot 0): **1 comparação**. - ID 7 (cauda do slot 0): **3 comparações**. - ID 99 (inexistente, mod 7 = 1, slot vazio): **0 comparações**.
5. Remoção e Reorganização	Remoção do ID `14` (nó do meio da colisão do slot `0`). Os ponteiros foram religados e a cadeia passou a `21 -> 7 -> null`. Fator de carga recalculado para \$\alpha = 0,57\$ (4/7).

6. Acesso Pós-Remoção	Nova busca pelo ID `7`. O número de comparações caiu de **3 para 2**, comprovando experimentalmente que a lista permaneceu íntegra e acessível pós-remoção.
-----------------------	---

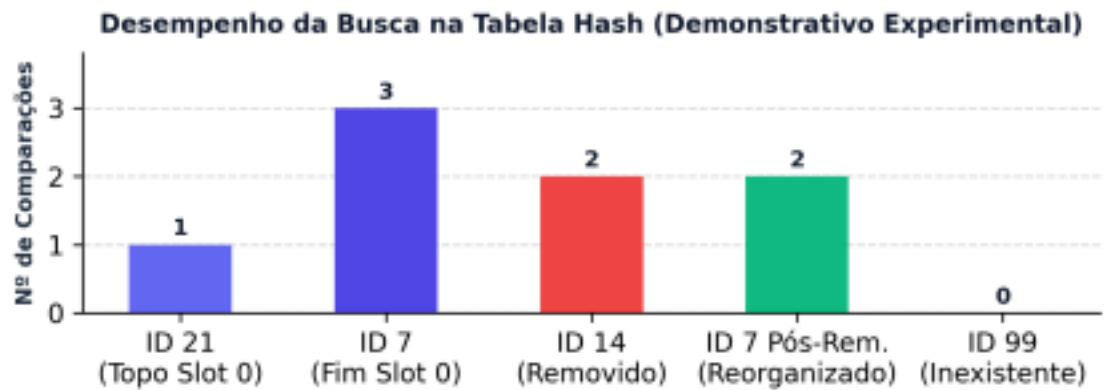


Figura 1: Gráfico comparativo do número de comparações executadas nas buscas de teste no terminal.

6. ANÁLISE DE EFICIÊNCIA ASSINTÓTICA (BIG-O) E FATOR DE CARGA

A eficiência da Tabela Hash é regida pelo **Fator de Carga** ($\alpha = N / M$), que representa a razão entre a quantidade de elementos armazenados (N) e o tamanho da tabela (M).

Inserção	$O(1)$	$O(N)$	Inserção com checagem de duplicadas quando todas as N chaves colidem no mesmo slot.
Busca	$O(1 + \alpha)$	$O(N)$	Todas as chaves colidem em um único índice, degradando a tabela em uma Lista Encadeada Linear.

Remoção	$O(1 + \alpha)$	$O(N)$	Localizar a chave no final da única lista colidida antes de desvincular os ponteiros.
---------	-----------------	--------	---

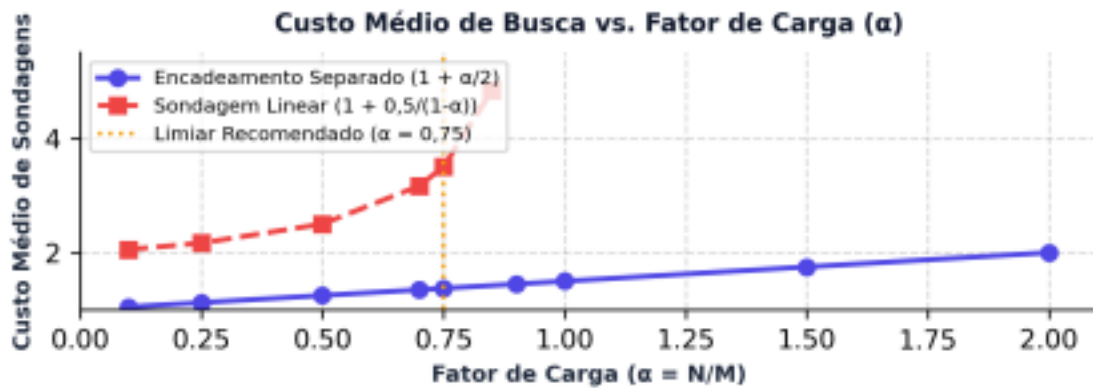


Figura 2: Curva teórica de degradação do número de sondagens/comparações em função do Fator de Carga (α).

7. MATRIZ COMPARATIVA DE ESTRATÉGIAS DE COLISÃO

Localização dos Dados	Vetor principal + Listas Encadeadas externas.	Exclusivamente no vetor principal da tabela.
Fator de Carga Máximo (α)	Pode ultrapassar 1.0 ($\alpha > 1$) sem transbordar.	Estritamente limitado a $\alpha < 1.0$. Falha ao encher.
Mecanismo de Exclusão	Simples: desvincula o nó alterando os elos da lista.	Exige o marcador de posição 'Lápide' (REMOVIDO)**.
Fenômeno de Agrupamento	Não sofre agrupamento no vetor (listas independentes).	Sofre de 'Agrupamento Primário' (Primary Clustering)**.
Uso de Memória	Alocação dinâmica sob demanda (ponteiro <code>proximo</code>).	Memória estática pré-alocada de tamanho fixo <code>MS</code> .

8. CONTRIBUIÇÃO DA EQUIPE, REFERÊNCIAS E TRANSPARÊNCIA DE IA

Divisão de Trabalho e Contribuição Individual:

jawc / Integrantes Guilda 3	Desenvolvimento completo das classes Java (Pergaminho, NoHash, TabelaHash, Main), criação dos casos de teste lúdico, validação no terminal, redação da documentação técnica e elaboração do LEIA-ME.	100%

Declaração Transparente do Uso de IA Generativa:

Em conformidade com a política de integridade acadêmica do projeto, registra-se que ferramentas de IA generativa foram utilizadas como **ferramenta de apoio secundária** para suporte na depuração de código, organização dos cenários de teste, e formatação da documentação em PDF. Toda a validação algorítmica, execução experimental e autoria intelectual das decisões de projeto foram realizadas de forma independente pelo grupo.

Referências Bibliográficas:

1. TENENBAUM, Aaron M.; LANGSAM, Yedidyah; AUGENSTEIN, Moshe J. *Estruturas de Dados Usando C*. São Paulo: Makron Books, 1995. Seção 7.4 ('Espalhamento').
2. KNUTH, Donald E. *The Art of Computer Programming, Vol. 3: Sorting and Searching*. Reading: Addison-Wesley, 1973.
3. FELLER, William. *An Introduction to Probability Theory and Its Applications*. New York: Wiley, 1968 (Paradoxo do Aniversário).
4. CAVALHEIRO, Marcos Ronaldo Melo. *Tabela Hash: O Acesso Direto por Transformação de Chave*. Material Didático de Estrutura de Dados (Aula 06), UNIJUÍ, 2026.
5. ORACLE. *Java SE 21 Documentation — java.util.HashMap*. Oracle Corporation, 2023.